

=====

PROJETO ArgusAI – MAPEAMENTO E NAVEGAÇÃO (Webots + ROS 2 Humble)

Registo de erros encontrados e respetivas correções

=====

OBJETIVO DO PROJETO

Robô TIAGO Lite (Webots) que:

- 1) mapeia um ambiente controlado (arena com um carro BMW X5 ao centro);
- 2) navega autonomamente até 4 pontos distintos à volta do carro (frente, trás, lado esquerdo, lado direito);
- 3) tira uma fotografia em cada ponto;
- 4) evita obstáculos (paredes, barris) e regressa ao ponto inicial.

Ferramentas: ROS 2 Humble, Webots R2025a, slam_toolbox (SLAM), Nav2 (navegação), RViz (visualização).

=====

FASE 1 – MAPEAMENTO

=====

ERRO 1 – O lidar não aparecia no RViz / o mapa não se construía

SINTOMA:

O SLAM não gerava mapa; o RViz não mostrava o laser.

CAUSA:

A árvore de transformações (TF) estava mal ligada. O comando usado publicava uma transformação entre dois nomes do próprio lidar ("Hokuyo_URG-04LX-UG01" -> Hokuyo_URG_04LX_UG01), mas NENHUM deles estava ligado ao "base_link" do robô. O slam_toolbox precisa de saber onde está o lidar em relação à base (base_link -> frame do lidar).

CORREÇÃO:

Publicar a TF correta: base_link -> Hokuyo_URG_04LX_UG01 (com offset de altura). Ficou integrado no launch próprio (argus_mapping_launch.py), juntamente com o relay do tópico do laser para /scan.

ERRO 2 – Condição de corrida (race condition) com o /scan

SINTOMA:

Por vezes o SLAM arrancava antes de existir o tópico /scan.

CAUSA:

O "relay" do laser era lançado manualmente, num terminal separado, depois do SLAM já estar a correr.

CORREÇÃO:

Integrar o relay e os publicadores de TF no próprio ficheiro de launch, todos sincronizados no arranque (waitForControllerConnection).

ERRO 3 – O robô não andava com o teclado (teleop)

SINTOMA:

O robô ficava parado mesmo a enviar comandos de velocidade.

CAUSA:

Existiam processos antigos de uma experiência anterior (nó extra a publicar em /cmd_vel) que interferiam com o controlo.

CORREÇÃO:

Limpar processos pendurados; conduzir publicando no tópico /cmd_vel normal (teleop_twist_keyboard). Regra adotada: antes de relançar, matar processos antigos de ROS/Webots.

ERRO 4 – "Detected jump back in time. Clearing TF buffer"

SINTOMA:

Logo no arranque dava o erro de "salto para trás no tempo".

CAUSA:

Processos ROS de uma sessão anterior ainda vivos. Ao reiniciar o Webots, o relógio da simulação volta a zero, mas os nós antigos tinham carimbos temporais mais altos -> o TF deteta um salto para trás.

CORREÇÃO:

Encerrar SEMPRE a simulação por completo (Ctrl+C) e limpar processos antes de relançar. Reiniciar o daemon do ROS 2 quando necessário.

ERRO 5 – "sequence size exceeds remaining buffer"

SINTOMA:

Mensagens de erro repetidas nos comandos de linha (ros2 topic echo/hz).

CAUSA:

Bug do middleware de comunicação FastDDS (transporte de memória partilhada) no Webots + ROS 2. NÃO tem relação com o tamanho do mapa.

NOTA IMPORTANTE (lição aprendida):

Tentámos corrigir com um perfil FastDDS "sem memória partilhada" (só UDP). Isso TIROU o erro, MAS introduziu um problema pior: um flood contínuo de "jump back in time" que fazia o slam_toolbox CRASHAR. Conclusão: esse perfil foi DESCARTADO.

CORREÇÃO FINAL:

Manter o FastDDS por omissão. O erro "sequence size" é apenas ruído cosmético da linha de comandos – NÃO afeta o mapeamento (o mapa constrói-se na mesma). Solução limpa opcional (não usada): instalar o middleware CycloneDDS.

ERRO 6 – Paredes a entortar / drift (o problema principal)

SINTOMA:

Ao início o mapa estava bom, mas a meio começava a "torcer" – as paredes deixavam de estar a direito.

CAUSA:

Erro acumulado da ODOMETRIA. O slam_toolbox, nesta configuração, tem o "scan matching" DESLIGADO (use_scan_matching: false) por causa de um bug do lidar do Webots (issue #5540), por isso confia 100% na odometria das rodas – qualquer erro vai direto para o mapa, sem correção.

TENTATIVA QUE NÃO RESULTOU:

Ligar o scan matching (use_scan_matching: true). Resultado: o slam_toolbox produzia um mapa PIOR (paredes duplicadas/esborratadas) porque o bug do lidar do Webots corrompe o scan matching. Mantido em false.

TENTATIVA QUE NÃO RESULTOU (2):

Alterar o basicTimeStep do mundo Webots (de 32ms para 20ms) para alinhar com o controlador (50Hz). Não melhorou de forma significativa; revertido.

CORREÇÃO QUE RESULTOU – Calibração da odometria:

A odometria configura-se no ficheiro ros2_control.yml (controlador diff_drive), NÃO no slam_toolbox. Calibrámos dois parâmetros medindo o movimento REAL com o lidar e comparando com o que a odometria reportava:

- a) wheel_separation_multiplier (erro de ROTAÇÃO):
 - método: rodar o robô e medir a rotação real por correlação dos scans do lidar (script calibrar_odom.py);
 - a odometria sub/sobre-estimava a rotação ~9%;
 - corrigido de 1.089 -> 0.987;
 - erro de rotação ficou ~0.4%.
- b) wheel_radius (erro LINEAR/distância):
 - método: apontar a uma parede, andar uma distância e medir a variação do alcance do lidar (script calibrar_linear.py);
 - já estava bom (erro ~0.3%);

- ajuste mínimo 0.0985 -> 0.0988.

RESULTADO:

Drift de rotação reduzido de ~9% para ~0.4%. As paredes passaram a ficar a direito. O erro residual que ainda se nota em percursos muito longos é uma limitação do lidar simulado (sem scan matching não há correção), minimizada conduzindo devagar e em curvas suaves.

LIMPEZA DO MAPA FINAL

PROBLEMA:

Pontos de ruído dispersos e uma parede ligeiramente duplicada (drift ao fechar a volta ao carro).

CORREÇÃO:

Removidos os componentes de ruído pequenos do mapa (.pgm) por análise de componentes ligados, mantendo as paredes e o carro intactos. Guardada cópia de segurança (map_backup.pgm) antes de editar.

FASE 2 – NAVEGAÇÃO (Nav2) E FOTOGRAFIAS

ERRO 7 – Coordenadas dos pontos no frame errado

SINTOMA:

Os 4 pontos de inspeção do navegacao.py iam dar a sítios errados.

CAUSA:

Os pontos estavam em coordenadas do MUNDO Webots, mas o Nav2 trabalha no frame do MAPA (origem = pose inicial do robô). No mapa, o carro está em ~(6.63, 4.68), não na posição do mundo.

CORREÇÃO:

Deteção automática do carro no mapa (obstáculo central) e cálculo dos 4 pontos à sua volta, virados para o carro e validados como espaço livre (script detetar_pontos.py). Pontos atualizados no navegacao.py.

ERRO 8 – Tópico da câmara errado

SINTOMA:

Não havia imagem da câmara.

CAUSA:

O navegacao.py subscrevia /Tiago_Lite/Astra/rgb/image_raw, que não existe. O tópico real é /Tiago_Lite/Astra_rgb/image_color.

CORREÇÃO:

Corrigido o nome do tópico no navegacao.py.

ERRO 9 – "ModuleNotFoundError: No module named 'paho'"

SINTOMA:

O navegacao.py crashava logo no arranque.

CAUSA:

A biblioteca paho-mqtt (paragem de emergência por MQTT) não estava instalada, e o import falhava.

CORREÇÃO:

Import do MQTT tornado opcional (try/except). A missão corre na mesma sem MQTT. (Alternativa: instalar com "pip install paho-mqtt".)

ERRO 10 – As fotografias saíam vazias

SINTOMA:

Chegava aos 4 pontos mas dizia "Não há imagem da câmara disponível" e

não guardava fotos.

CAUSA:

Durante a navegação, o nó da missão NÃO estava a ser "spun" (rclpy.spin_once), por isso o callback da câmara nunca era chamado e a imagem ficava sempre vazia. O loop de espera só fazia time.sleep().

CORREÇÃO:

Substituir os time.sleep() por rclpy.spin_once(self, ...) nos loops de navegação, e dar alguns "spins" extra antes de capturar, para garantir um frame fresco da câmara.

ERRO 11 – Localização (AMCL) perdia-se / robô não parava na base

SINTOMA:

Depois de dar a volta, o robô parecia "perder-se" no RViz no caminho de volta ao ponto inicial.

CAUSA:

Erro de odometria acumulado na longa viagem de regresso, com o mapa imperfeito de um dos lados.

CORREÇÃO:

Reiniciar a navegação do zero (AMCL re-inicializa a pose em 0,0,0, que coincide com a pose inicial do robô – set_initial_pose: True). Garantir paragem do robô no fim (travar_robô). Conduzir/regressar com a odometria já calibrada resolveu o problema.

=====

OTIMIZAÇÃO DO PERCURSO – A* + CAIXEIRO-VIAJANTE (TSP)

=====

PROBLEMA – Percurso ineficiente / regresso "pelo caminho que veio"

SINTOMA:

O robô ziguezagueava entre os 4 pontos e, no regresso, parecia voltar por um caminho longo em vez do mais rápido até ao ponto inicial.

ESCLARECIMENTO IMPORTANTE (dois problemas diferentes):

Há que distinguir duas coisas que costumam confundir-se:

- (1) Encontrar o caminho mais curto ENTRE DOIS pontos -> algoritmo A*.
- (2) Escolher a melhor ORDEM de visita de VÁRIOS pontos -> problema do caixeiro-viajante (TSP, Traveling Salesman Problem).

O caminho de cada troço já era ótimo (o Nav2 planeia-o). O que estava mal era a ORDEM dos pontos -> isso é TSP, não A*.

SOLUÇÃO 1 – A* no planeador global do Nav2

O planeador global do Nav2 (NavfnPlanner) estava com "use_astar: false" (usava Dijkstra). Ativámos o A*:

use_astar: true (em nav2_params.yaml, cópia local)

A* usa uma heurística (distância em linha reta ao objetivo) para explorar menos células e calcular o caminho mais depressa. Encontra o MESMO caminho ótimo que o Dijkstra, mas com menos cálculo.

(Nota: o A* só acelera o cálculo; não muda o facto de cada troço já ser o mais curto. Por isso, sozinho, não resolvia o problema do percurso.)

SOLUÇÃO 2 – Caixeiro-viajante (TSP) para a ORDEM dos pontos

O problema do caixeiro-viajante consiste em, dado um conjunto de pontos, encontrar a ORDEM de visita que minimiza a distância total do percurso (aqui: começar e acabar no ponto inicial, passando pelos 4 pontos).

Resolução (script tsp_astar.py):

- a) Usámos o A* sobre a grelha do mapa para medir a distância REAL entre cada par de pontos (incluindo o ponto inicial), já a contornar o carro e as paredes. Isto dá uma matriz de distâncias correta (a distância em linha reta seria errada, pois atravessaria o carro).

- b) Como são só 4 pontos, testámos por força bruta todas as 24 ordens possíveis e escolhemos a de menor distância total.

RESULTADO:

- Ordem ingénua (original): ~41 m, com ziguezague (ia ao ponto mais longe primeiro, voltava atrás, etc.).
- Ordem ótima (TSP): ~28 m – um ANEL à volta do carro
base -> direita -> frente -> esquerda -> trás -> base
Poupa ~13 m, não cruza para trás, e termina no ponto mais próximo da base (regresso curto e direto).

Em resumo: o A* (Solução 1) acelera o cálculo de cada caminho e foi usado também para medir distâncias; o TSP (Solução 2) é o que realmente eliminou o ziguezague e o regresso longo, ao otimizar a ORDEM de visita.

ERRO 12 – O robô andava às voltas no objetivo e não parava

SINTOMA:

Ao chegar a um ponto (sobretudo no regresso à base), o robô ficava a rodar sem fim, sem assentar e sem parar.

CAUSA:

Comportamento INTERMITENTE do controlador local do Nav2 (DWB). O ponto inicial (0,0) fica num canto apertado, perto dos barris. Aí o DWB por vezes não consegue gerar uma trajetória válida (erro "scoring function failed to prepare") e entra em recuperação rotacional (spin) em ciclo. É intermitente porque depende da aleatoriedade do AMCL (filtro de partículas), do ângulo exato de chegada e do ruído do lidar nesse instante – numa corrida assentava, noutra ficava a rodar.

NOTA: NÃO foi causado pela mudança da ordem dos pontos nem pela config do RViz (uma corrida com a ordem nova já tinha voltado bem à base).

CORREÇÃO (tornar robusto em vez de depender da sorte):

1. Tolerâncias do objetivo mais largas (nav2_params.yaml):
xy_goal_tolerance 0.25 -> 0.30 ; yaw_goal_tolerance 0.25 -> 0.50
(assenta mais cedo, menos "afinação" fina que provocava o spin).
2. Inflação dos obstáculos reduzida: inflation_radius 0.55 -> 0.45
(o canto da base fica mais navegável, o DWB falha menos).
3. Timeout por objetivo no navegaçao.py (35 s): se ficar preso, cancela, para o robô e segue/termina – nunca mais fica a rodar para sempre.
Além disso, a foto é tirada na mesma se parou perto do ponto.

RESULTADO:

Os 4 pontos passaram a ser alcançados sem spin e o regresso à base termina com o robô PARADO no ponto inicial.

=====

RESULTADO FINAL

=====

Pipeline completo a funcionar de ponta a ponta (corrida final validada):

- Mapeamento com odometria calibrada -> mapa limpo e utilizável;
- Navegação Nav2 aos 4 lados do carro com desvio de obstáculos;
- Ordem de visita otimizada (TSP + A*): base -> dir -> frente -> esq -> tras -> base (anel a volta do carro, ~28 m em vez de ~41 m);
- Planeador global a usar A* (use_astar: true);
- 4 fotografias guardadas (foto_carro_dir/frente/esq/tras.jpg), cada uma a mostrar o respetivo lado do carro;
- Regresso ao ponto inicial com o robô a PARAR (terminou em ~(0.3, 0.0), dentro da tolerancia de 0,0);
- Visualização em RViz com mapa, costmaps, caminho global/local, partículas do AMCL e rasto do percurso (config nav.rviz).

PRINCIPAIS LIÇÕES APRENDIDAS:

1. O drift do mapa vinha da ODOMETRIA, não do SLAM – corrige-se em ros2_control.yml (parâmetros das rodas), não no slam_toolbox.
2. Nem todas as "correções" ajudam: o perfil FastDDS sem memória partilhada e o scan matching pioraram as coisas e foram descartados.
3. Confirmar SEMPRE os nomes reais dos tópicos (câmara, laser).
4. Em ROS 2, um nó só processa callbacks quando é "spun".
5. Limpar processos antigos antes de relançar evita erros de TF/tempo.
6. Alguns problemas do Nav2 são INTERMITENTES (ex.: spin no objetivo por

aleatoriedade do AMCL). A solucao e tornar o sistema robusto (tolerancias, timeouts), nao procurar uma causa unica.

```
=====
FICHEIROS PRINCIPAIS
=====
```

```
argus_mapping_launch.py  - launch do mapeamento (SLAM)
argus_nav_launch.py      - launch da navegacao (Nav2)
ros2_control.yml         - odometria calibrada (rodas)
slam_toolbox_params.yaml - parametros do SLAM
navegacao.py             - missao (4 pontos + fotos + regresso)
detetar_pontos.py         - calcula os 4 pontos no frame do mapa
tsp_astar.py             - resolve o caixeiro-viajante com A* (ordem otima)
nav2_params.yaml         - parametros do Nav2 (A*, tolerancias, inflacao)
nav.rviz                 - config do RViz para navegacao (caminho, rasto)
calibrar_odom.py         - calibracao da rotacao (wheel_separation_multiplier)
calibrar_linear.py       - calibracao linear (wheel_radius)
map.pgm / map.yaml       - mapa final (map_backup.* = copia de seguranca)
foto_carro_{dir,frente,esq,tras}.jpg - fotografias dos 4 lados do carro
```